GCoM: A Detailed GPU Core Model for Accurate Analytical Modeling of Modern GPUs

Jounghoo Lee
Yonsei University
Seoul, South Korea
jounghoolee@yonsei.ac.kr

Yeonan Ha
Yonsei University
Seoul, South Korea
yeonan@yonsei.ac.kr

Suhyun Lee
Yonsei University
Seoul, South Korea
su_hyun@yonsei.ac.kr

Jinyoung Woo
Ajou University
Suwon, South Korea
woo96923@ajou.ac.kr

Jinho Lee
Yonsei University
Seoul, South Korea
leejinho@yonsei.ac.kr

Hanhwi Jang
Ajou University
Suwon, South Korea
hanhwi@ajou.ac.kr

Youngsok Kim*
Yonsei University
Seoul, South Korea
youngsok@yonsei.ac.kr

ABSTRACT

Analytical models can greatly help computer architects perform orders of magnitude faster early-stage design space exploration than using cycle-level simulators. To facilitate rapid design space exploration for graphics processing units (GPUs), prior studies have proposed GPU analytical models which capture first-order stall events causing performance degradation; however, the existing analytical models cannot accurately model modern GPUs due to their outdated and highly abstract GPU core microarchitecture assumptions. Therefore, to accurately evaluate the performance of modern GPUs, we need a new GPU analytical model which accurately captures the stall events incurred by the significant changes in the core microarchitectures of modern GPUs.

We propose *GCoM*, an accurate GPU analytical model which faithfully captures the key core-side stall events of modern GPUs. Through detailed microarchitecture-driven GPU core modeling, GCoM accurately models modern GPUs by revealing the following key core-side stalls overlooked by the existing GPU analytical models. First, GCoM identifies the compute structural stall events caused by the limited per-sub-core functional units. Second, GCoM exposes the memory structural stalls due to the limited banks and shared nature of per-core L1 data caches. Third, GCoM correctly predicts the memory data stalls induced by the sectored L1 data caches which split a cache line into a set of sectors sharing the same tag. Fourth, GCoM captures the idle stalls incurred by the inter- and intra-core load imbalances. Our experiments using an NVIDIA RTX 2060 configuration show that GCoM greatly improves the modeling accuracy by achieving a mean absolute error of 10.0% against Accel-Sim cycle-level simulator, whereas the state-of-the-art GPU analytical model achieves a mean absolute error of 44.9%.

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISCA '22, June 18–22, 2022, New York, NY, USA
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-8610-4/22/06…$15.00
https://doi.org/10.1145/3470496.3527384

CCS CONCEPTS

• **Computing methodologies** → **Modeling methodologies**; **Graphics processors**.

KEYWORDS

graphics processing units, performance modeling, interval analysis

ACM Reference Format:
Jounghoo Lee, Yeonan Ha, Suhyun Lee, Jinyoung Woo, Jinho Lee, Hanhwi Jang, and Youngsok Kim. 2022. GCoM: A Detailed GPU Core Model for Accurate Analytical Modeling of Modern GPUs. In *The 49th Annual International Symposium on Computer Architecture (ISCA '22), June 18–22, 2022, New York, NY, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3470496.3527384

1 INTRODUCTION

By replacing long-running cycle-level simulators with a performance model, computer architects can rapidly identify and analyze the performance-degrading stall events of their applications and hardware [15, 16, 27, 32, 43, 47]. Among various types of performance models, analytical models have been shown to achieve high modeling accuracy and provide a deep insight into the causes of the stall events [14]. Analytical models can not only predict the performance but also construct cycles-per-instruction (CPI) stacks. CPI stacks provide a breakdown of the execution latency into various types of the stall events, allowing computer architects to identify the key performance bottlenecks of their applications and hardware.

To enable rapid design space explorations of graphics processing unit (GPU) architectures, prior studies [23, 25, 50] have proposed GPU analytical models which capture the performance characteristics unique to GPUs. For example, GPUMech [25] focuses on capturing the warp-driven highly-threaded execution model of GPU architectures. Its multi-warp model takes into account the scheduling policy of the warp schedulers and uses an M/D/1 queue to model the off-chip DRAM bandwidth. As another example, MDM [50] targets memory-divergent GPU applications which exhibit poor spatial locality and incur high memory-side contention.

Aimed at achieving higher computational throughputs, modern GPUs have introduced several enhancements to their core microarchitectures. However, the enhancements can incur core-side stall events which significantly lower the performance. First, modern GPU cores partition their computational resources into sub-cores, each of which consists of a single-issue warp scheduler and a set

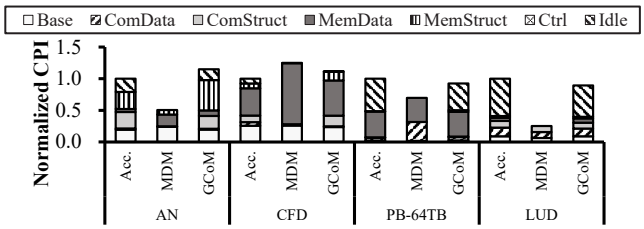


Figure 1: The CPI stacks constructed by Accel-Sim, MDM, and GCoM (ours) with an NVIDIA RTX 2060 configuration

of functional units having limited lane counts. The limited functional units cause *compute structural stalls* as they prevent a warp scheduler from issuing one warp instruction per cycle. Second, the per-core L1 data caches (D$s) incur *memory structural stalls* as each L1 D$ has limited banks and is now shared by the sub-cores. The limited L1 D$ banks make it difficult for a warp to fetch its data with a single L1 D$ access, and sharing an L1 D$ between sub-cores induces contention. Third, modern GPU cores employ sectored L1 D$s which split each of their cache lines into smaller sectors sharing the same tag (e.g., a 128-byte line into four 32-byte sectors). The sectored L1 D$s have a large impact on *memory data stalls* by accessing the L2 $ at the sector granularity instead of the cache line granularity. Fourth, as modern GPUs have high core counts and per-core sub-core counts, the load imbalances between the cores and between the sub-cores of the same core occur more frequently. The load imbalances result in intra- and inter-core *idle stalls*. To provide accurate insights into the performance by capturing these key core-side stall events, GPU analytical models should faithfully model the microarchitectural characteristics of modern GPU cores.

Unfortunately, the existing GPU analytical models suffer from high modeling errors with modern GPUs due to their outdated and highly abstract GPU core microarchitecture assumptions. They cannot capture the key core-side stall events by assuming that the GPU cores do not get partitioned into sub-cores, have a sufficient amount of functional unit and L1 D$ resources, and have the same amount of work. Fig. 1 depicts the high modeling errors of MDM [50], the state-of-the-art GPU analytical model, due to its lack of detailed modern GPU core modeling. The CPI stacks constructed with Accel-Sim cycle-level GPU simulator [30] demonstrate that the four benchmarks are bottlenecked by the key core-side stalls: `AN` by the compute and memory structural stalls, `CFD` by the memory structural stalls, and `PB` and `LUD` by the idle stalls due to intra- and inter-core load imbalances, respectively. However, MDM overlooks the core-side stalls due to its abstract GPU core model, and thus provides inaccurate insights into the performance, forcing computer architects to rely on long-running cycle-level simulators for accurate design space explorations. Therefore, we need a new GPU analytical model which accurately captures the core-side stall events to enable rapid and accurate GPU design space explorations.

In this paper, we present *GCoM*, a detailed GPU core model which faithfully models the core microarchitecture characteristics of modern GPUs. We make a key finding that, to achieve accurate modern GPU core modeling, hierarchical modeling of modern GPU cores driven by their microarchitecture characteristics is necessary. Motivated by the key finding, GCoM employs a set of analytical expressions which model the limited functional units of each sub-core, the contention and sectors of each per-core L1 D$, and the intra/inter-core load imbalances in a bottom-up manner. GCoM captures the stalls occurring at different hierarchies of modern GPU cores: the compute structural stalls at the sub-core level, the memory structural/data and intra-core idle stalls at the core level, and the inter-core idle stalls at the GPU level. The major components of GCoM are a sub-core model, an intra-SM contention model, a sectored/streaming cache simulator, and an inter-core load imbalance model. Using its detailed GPU core model, GCoM accurately models the performance of modern GPUs and reveals the key core-side stalls overlooked by the existing GPU analytical models.

We evaluate GCoM by comparing its performance predictions with MDM, the state-of-the-art GPU analytical model, and by using Accel-Sim, a detailed cycle-level simulator validated against modern NVIDIA GPUs, as the ground truth. Our experiments using 31 GPU applications as benchmarks and an NVIDIA RTX 2060 GPU configuration show that GCoM greatly improves the modeling accuracy by achieving a mean absolute error (MAE) of 10.0%, whereas MDM falls short by achieving an MAE of 44.9%. GCoM also achieves an MAE of 24.4% against a real NVIDIA RTX 2060 GPU. The results clearly demonstrate the high importance and effectiveness of GCoM's detailed modern GPU core modeling, and that GCoM should be favored over the existing models for fast and accurate modern GPU design space explorations.

To summarize, this paper makes the following contributions:

- We identify that the outdated GPU core microarchitecture assumptions of the existing GPU analytical models provide incorrect insights into the performance of modern GPUs.
- We design and propose GCoM, a detailed and accurate GPU core model which faithfully captures the key core-side stall events of modern GPUs through hierarchical microarchitecture-driven GPU core modeling.
- GCoM accurately predicts the performance of modern GPUs by achieving an MAE of 10.0% against Accel-Sim with an NVIDIA RTX 2060 configuration.
- We demonstrate the high effectiveness of GCoM toward fast and accurate design space explorations by showing that GCoM can construct accurate CPI stacks with various GPU configurations.

2 BACKGROUND & MOTIVATION

2.1 GPU Architectures

Graphics processing units (GPUs) employ a highly multi-threaded execution model which can concurrently execute up to tens of thousands of threads at a time. Fig. 2 depicts the microarchitecture of the NVIDIA Turing GPU [9], a modern GPU architecture introduced in 2018. A GPU consists of multiple streaming multiprocessors (SMs), an interconnection network (NoC), and multiple memory channels. An SM is partitioned into multiple *sub-core*s, each of which consists of a warp scheduler, functional units capable of executing different types of operations, a load/store (LD/ST) unit, a register file, and an L0 instruction cache (I$). The SM is also equipped with a write-no-allocate L1 data cache (D$) and an L1 I$. The L1 D$ is lockup-free (streaming) and sectored; a 128-byte cache line is split into four 32-byte sectors which share the same tag. The load/store unit of each sub-core interacts with the L1 D$ to load/store data from/to

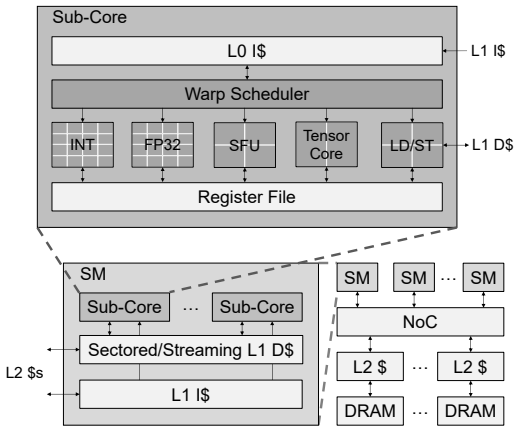
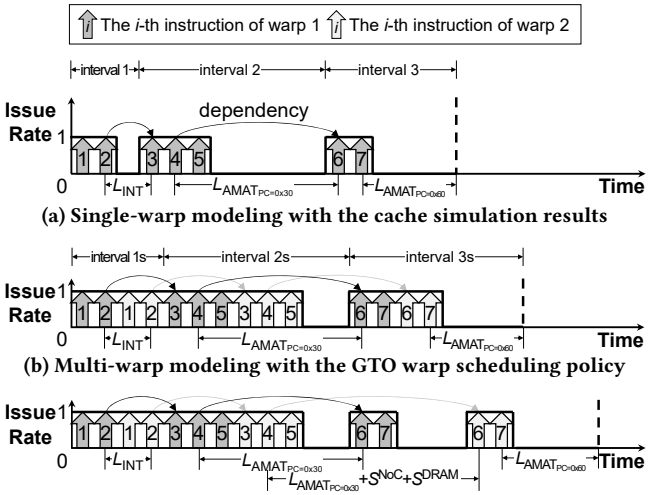


Figure 2: The microarchitecture of the NVIDIA Turing GPU



(a) Single-warp modeling with the cache simulation results

(b) Multi-warp modeling with the GTO warp scheduling policy

(c) Multi-warp modeling with the memory-side contention

Figure 3: GPU analytical modeling based on interval analysis

the memory channels. Each memory channel has a write-back L2 cache partition and an off-chip DRAM partition. The GPU interleaves its physical memory space across the memory channels at a fine granularity of 256 bytes. The NoC connects the SMs and the memory channels.

Applications execute their operations on a GPU by invoking kernels written with GPU programming APIs (e.g., CUDA [18], OpenCL [34, 45]). A kernel groups its threads into thread blocks (TBs). The threads within the same TB can synchronize their progress and share data through shared memory. When an application invokes a kernel, a GPU assigns a TB to an SM having sufficient free resources (e.g., registers). Then, the SM further splits the TB into warps (32 threads per warp on NVIDIA GPUs) and distributes the warps to its sub-cores. The threads of the same warp get executed in a lock-step manner; if the threads encounter a branch divergence, the execution of the threads following one path is time-interleaved with the execution of the other threads. After that, the warp scheduler of each sub-core issues one of the assigned warps to the functional and LD/ST units with a maximum throughput of one warp instruction per clock. As each warp scheduler can issue up to one warp instruction per cycle, the maximum number of warp instructions the SM can execute per cycle is equivalent to the number of warp schedulers of the SM.

2.2 GPU Analytical Models

Aimed at providing a deep insight into GPU performance, prior studies [25, 50, 51] have proposed GPU analytical models based on interval analysis, a well-known approach for accurately modeling CPU performance [14, 28]. The key idea of modeling the performance with interval analysis is that a warp scheduler can sustain its maximum issue rate when no stall events occur. Ideally, a warp scheduler should sustain a throughput of one warp instruction per cycle throughout the execution of a kernel; however, the warp scheduler stalls if it encounters one of the following stall events: (a) there is no warp assigned to the warp scheduler (e.g., the warp scheduler finished executing all of its assigned warps), (b) there is no ready warp due to unresolved data/control dependencies and/or TB-wide barriers, or (c) there is a ready warp, but its target functional or LD/ST unit has not completed executing its previous warp instruction. As there are various stall events which can prevent a GPU from achieving its peak performance, a GPU analytical model should accurately capture the stall events and estimate their impacts on the performance.

To accurately capture the warp-based execution model of GPUs, the existing GPU analytical models employ a cache simulator, a single-warp model, a multi-warp model, and a memory-side contention model. Fig. 3 shows how the existing models estimate the performance in a bottom-up manner. First, the models select a representative warp and construct its execution timeline using the functional unit latencies and the average memory access times (AMATs) of the load and store instructions estimated by the cache simulator. The prior studies [25, 50] have shown that the warps of a kernel exhibit similar execution behaviors, and thus estimating the performance by abstracting the warps as the representative warp can achieve high modeling accuracy. The cache simulator takes as input the memory instructions from each warp in a round-robin manner. Second, the models split the execution timeline of the representative warp into intervals using their single-warp models (Fig. 3a). Each interval consists of a sequence of instructions achieving the maximum issue rate (i.e., one warp instruction per cycle) followed by the stall cycles induced by various stall events. Third, the models derive the per-SM execution timeline using their multi-warp models which apply the warp scheduling policy, e.g., greedy-then-oldest (GTO) [42], and assume that the SM gets populated with the representative warp (Fig. 3b).

After that, the models take into account the memory-side contention incurred by the limited number of miss-status holding registers (MSHRs) and/or the NoC and DRAM congestions (Fig. 3c). For example, MDM [50] focuses on the memory request batching due to the MSHR contention. MDM counts the L1 D$ cache read misses during the execution of the concurrently-issued intervals (e.g., interval 2s in Fig. 3b). Under the assumption that those read requests are sent to the NoC in a batch until the MSHR becomes full, the number of batches is the quotient of the request count and the MSHR size. Then, the amount of MSHR contention is the

Table 1: Characteristics of modern NVIDIA GPUs' SMs

GPU	Per-SM Resources		
	Sub-cores		L1 D$
	Count	Per-Sub-Core Resources	
Pascal [17]	2	Dual-issue warp scheduler, 32 FP32, 16 FP64, 8 SFU	Sectored, streaming; 128 B/line, 32 B/sector; 2 banks
Volta [12]	4	Single-issue warp scheduler, 16 INT, 16 FP32, 8 FP64, 4 SFU, 2 Tensor Cores	Sectored, streaming; 128 B/line, 32 B/sector; 4 banks
Turing [9]	4	Single-issue warp scheduler, 16 INT, 16 FP32, 4 SFU, 2 Tensor Cores	Sectored, streaming; 128 B/line, 32 B/sector; 4 banks
Ampere [38]	4	Single-issue warp scheduler, 16 INT+FP32, 16 FP32, 4 SFU, 1 Tensor Core	Sectored, streaming; 128 B/line, 32 B/sector; 4 banks

service time of all batches including the NoC/DRAM queuing delay. NoC and DRAM queuing delay of a batch is evaluated assuming all SMs identically send a batch of read request and write requests. With the mount of concurrent memory request, average L2 cache miss rate, and max bandwidth of NoC/DRAM, MDM estimates the queuing delays. Lastly, the models use the resulting multi-warp execution timeline to predict the GPU performance.

2.3 Limitations

In this work, we employ the GPU stall classification mechanism proposed in [2]. The following list shows a summary of the GPU stalls we consider for constructing CPI stacks:

- **Base**: The (sub-)core issued a ready warp.
- **Compute Data** (ComData): There is no ready warp due to the unresolved data hazards on functional unit outputs.
- **Compute Structural** (ComStruct): The (sub-)core cannot issue a ready warp as the target functional units have not completed executing their previous warp instructions.
- **Memory Data** (MemData): There is no ready warp due to the unresolved data hazards on memory accesses.
- **Memory Structural** (MemStruct): The (sub-)core cannot issue a ready warp due to the memory-side contentions.
- **Control/Synchronization** (Ctrl): There is no ready warp due to control hazards (e.g., TB-wide synchronizations).
- **Idle**: The (sub-)core has no warp to execute.

Aimed at achieving higher hardware utilizations and computational throughputs, GPU vendors have introduced several enhancements to their GPU core microarchitectures; however, the enhancements can trigger a number of core-side stall events which may greatly lower the GPU performance. Table 1 lists a few key core microarchitecture characteristics of modern NVIDIA GPUs. We find four potential core-side stall events which can become significant due to the enhancements. First, each sub-core can incur ComStruct stalls as it now takes more than one cycle to execute a warp instruction due to the limited functional units. Second, as each per-core L1 D$ is shared between multiple sub-cores and has limited banks, the L1 D$s may induce contention which results in MemStruct stalls. Third, the sectored L1 D$s can significantly alter the MemData stalls by splitting each of their cache lines into a set of smaller sectors sharing the same tag and by accessing the L2 $ and DRAM at the sector granularity. Fourth, as warps get distributed to a number of cores and sub-cores, the Idle stalls can occur due to intra- and inter-core load imbalances. As the core-side stalls induced by the enhancements in modern GPU cores can greatly lower the performance, GPU analytical models should faithfully

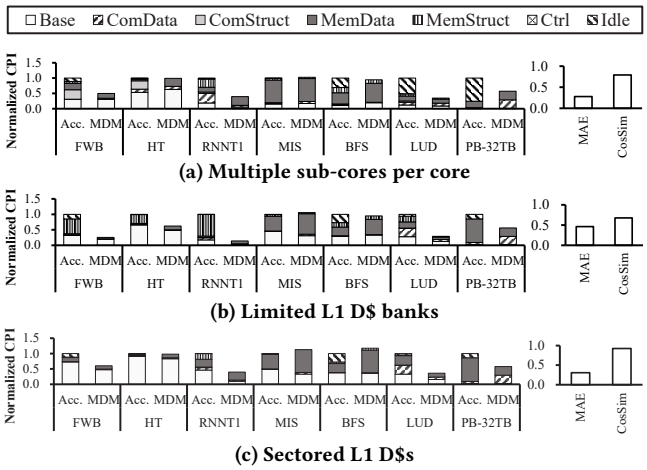


Figure 4: The high modeling errors of MDM with the key characteristics of the Turing GPU's SM microarchitecture

capture them to provide accurate insights into the performance. Especially, the insights provided by the analytical models should be compatible with those from modern detailed cycle-level simulators (e.g., Accel-Sim [30]) which recognize the high importance of accurate GPU core modeling and implement detailed timing models based on modern GPU core microarchitectures.

Unfortunately, we find that the existing GPU analytical models overlook the key core-side stall events. The existing models assume an outdated and highly abstract GPU core microarchitecture which cannot capture the key core-side stall events occurring at different microarchitectural levels in modern GPU cores. First, the existing models do not take into account the sub-cores, and thus fail to reveal the ComStruct stalls due to the limited per-sub-core functional units and the Idle stalls incurred by the load imbalances between the sub-cores of the same core. Second, at the core level, the existing models are unaware of the sectored L1 D$s and their impacts on the memory access patterns, making them difficult to correctly estimate the MemData stalls affected by the sectored L1 D$s. Third, the existing models cannot identify the idle stalls at the GPU level as they do not consider inter-core load imbalances. The lack of detailed hierarchical GPU core microarchitecture modeling prevents the existing models from capturing the key core-side stall events.

To quantify the importance of capturing the key core-side stall events, we examine how much impact the enhancements in modern GPU core microarchitectures have on the performance. We also analyze the impact of the enhancements on the modeling accuracy of MDM [50, 51], the state-of-the-art GPU analytical model. In this experiment, we configure Accel-Sim cycle-level simulator to simulate the simplified GPU core assumed by MDM (i.e., no sub-cores, 32 lanes per functional unit and 32 L1 D$ banks, and non-sectored L1 D$s). Then, we apply one of the three major microarchitectural enhancements of modern GPU cores (i.e., 4 sub-cores per core, 4 L1 D$ banks, and sectored L1 D$s) on top of the baseline MDM-friendly Accel-Sim configuration and compare the CPI stacks constructed by MDM and Accel-Sim.

Fig. 4 illustrates that the key core-side stall events can become a major performance bottleneck in various applications. It also shows

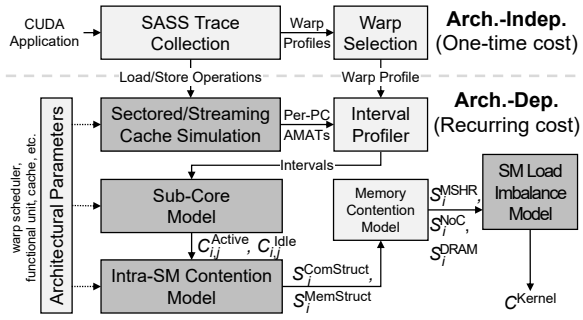


Figure 5: The overall architecture of GCoM

that MDM achieves high modeling errors with the enhancements, and thus provides inaccurate insights into the performance. First, Fig. 4a shows the performance impacts of partitioning a core into four sub-cores. The CPI stacks of Accel-Sim show that some benchmarks suffer from the ComStruct stalls due to the limited lanes of the per-sub-core functional units; however, MDM assumes sufficient functional unit resources and does not accurately predict the ComStruct stalls. Second, as we reduce the per-core L1 D$ banks from 32 to 4, the increased MemStruct stalls can be observed from Accel-Sim's CPI stacks (Fig. 4b). MDM, on the other hand, assumes sufficient L1 D$ banks, and thus cannot capture the MemStruct stalls. Third, introducing the sectored L1 D$ on top of the baseline configuration affects the MemData stalls as can be observed from Accel-Sim's CPI stacks (Fig. 4c). MDM's CPI stacks, however, tend to overestimate the MemData stalls as it does not model the sectors and assumes that the L1 D$s access the L2 $ and DRAM at their cache line granularity. Fourth, the CPI stacks of Accel-Sim also reveal the Idle stalls incurred by the intra- and inter-core load imbalances. Again, MDM cannot capture the Idle stalls as it assumes that an equal amount of work gets assigned to all the (sub-)cores. The results clearly demonstrate that accurate performance modeling of modern GPUs demands capturing the core-side stall events, and that the existing GPU analytical models cannot capture the core-side stall events due to their highly abstract GPU core model.

3 GCOM: A DETAILED GPU CORE MODEL

3.1 Overview

Motivated by the high modeling errors of the existing GPU analytical models with modern GPUs, we propose *GCoM*, a detailed GPU core model which faithfully captures the key core-side stalls of modern GPUs. Fig. 5 illustrates the overall architecture of GCoM. GCoM takes as input a GPU application and the architectural parameters of a target GPU. Then, GCoM collects the dynamic SASS instruction trace of the application through instrumentation on real GPUs using NVBit [49]. We focus on the dynamic traces as collecting them from a real GPU is much faster than collecting the intermediate PTX instructions [39] from a functional simulator. The SASS traces also allow GCoM to achieve higher modeling accuracy than using the PTX instructions [22]; however, when necessary, GCoM can be configured to use the PTX instructions instead. GCoM takes advantage of MDM's accurate memory contention modeling by employing MDM as its memory contention model.

Table 2: The one-time and recurring costs of GCoM compared to those of Accel-Sim cycle-level simulator

Design Points	Accel-Sim		GCoM		Speedup
	One-Time	Recurring	One-Time	Recurring	
1	7.9 hours	3.5 days	11.8 hours	13.8 minutes	7.6×
10		34.7 days		2.3 hours	59.7×
100		11.6 months		23.0 hours	240.0×
1,000		9.6 years		9.6 days	345.1×

After collecting the SASS traces, GCoM selects the representative warp of a kernel based on the per-warp profiles. GCoM, similar to the prior studies [25, 50], uses the representative warp to approximate the behaviors of the other warps. Such an approximation has been shown to achieve high modeling accuracy [25, 50]. The warp profiles reflect the architecture-independent characteristics of their corresponding warps (e.g., # of dynamic instructions, the distribution of the types of the instructions). To select the representative warp, GCoM performs k-means clustering on the warp profiles similar to GPUMech [25].

GCoM then performs a cache simulation using the collected load/store instructions and the provided architectural parameters. The interval profiling stage uses the cache simulation output (e.g., per-load/store instruction AMATs) to split the execution of the representative warp into intervals. The intervals, the cache simulation outputs, and the architectural parameters are then used by the following key components of GCoM: the sub-core model, the intra-SM contention model, and the SM load imbalance model. First, the sub-core model estimates the per-sub-core execution cycles using the intervals. Then, the intra-SM contention model predicts the per-SM execution cycles and the intra-SM contention cycles. After that, GCoM employs MDM to calculate the memory contention cycles. Lastly, the SM load imbalance model of GCoM captures the idle cycles incurred by the inter-SM load imbalances.

The trace collection and representative warp selection stages of GCoM incur a one-time cost for each application. The other components of GCoM incur a recurring cost which is proportional to the number of evaluated design points. Table 2 compares the costs of GCoM and Accel-Sim cycle-level simulator when evaluating various design points with our 31 benchmarks on a server equipped with an AMD EPYC 7713P CPU, an NVIDIA RTX 2060 GPU, and 1-TB DDR4 DRAM. For Accel-Sim, the one-time cost is the SASS trace collection overhead and the recurring cost is the time it takes to simulate the benchmarks. Despite having a larger one-time cost, GCoM can perform significantly faster design space explorations than Accel-Sim by incurring a low recurring cost.

3.2 Estimating the Total Cycles of a Kernel

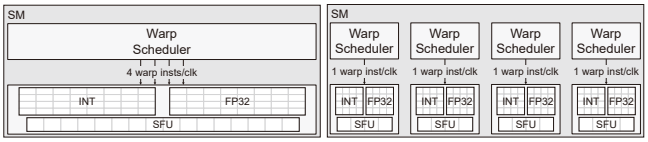
GCoM constructs the cycle stack of a kernel, C^{Kernel}, by averaging the execution cycles of the SMs of the GPU:

$$C^{Kernel} = \frac{\sum_{i=0}^{numSMs-1} C_i}{numSMs}$$

where C_i denotes the total cycles of the i-th SM.

To capture the Idle stalls caused by inter-SM load imbalances, GCoM breaks down the execution cycles of the i-th SM into *active* and *idle* cycles:

$$C_i = C_i^{Active} + C_i^{Idle}$$



(a) GPUMech and MDM (b) The NVIDIA Turing GPU

Figure 6: Differences in the core microarchitectures of the existing GPU analytical models and modern GPUs

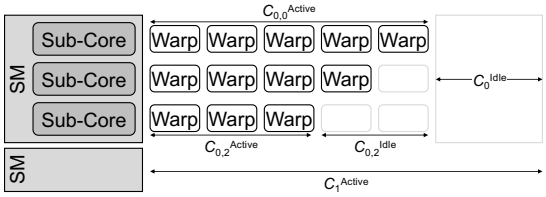


Figure 7: Modeling the intra- and inter-core load imbalances

where C_i^{Active} and C_i^{Idle} represent the active and idle cycles of the i-th SM, respectively. An SM is active when it is executing one or more TBs. When an active SM completes executing its TBs, it becomes idle and waits for all the other active SMs to become idle.

3.3 Modeling the Cycles of a Sub-Core

The existing GPU analytical models abstract the sub-cores as a single multi-issue warp scheduler (Fig. 6a). However, such an abstraction cannot capture the ComStruct stalls due to the limited functional units per sub-core and the Idle stalls incurred by a load imbalance between the sub-cores. Therefore, to accurately identify the ComStruct and Idle stalls, GCoM needs to faithfully model the sub-cores and their limited functional units as shown in Fig. 6b.

As the first step, GCoM partitions a sub-core's total cycles, $C_{i,j}$ into active and idle cycles to reveal the Idle stalls incurred by a load imbalance between the sub-cores:

$$C_{i,j} = C_{i,j}^{Active} + C_{i,j}^{Idle}$$

The Idle stalls account for the intra-core load imbalances, so GCoM computes them as the difference between the active cycles of one sub-core and the active cycles of the longest-running sub-core belonging to the same SM (Fig. 7):

$$C_{i,j}^{Idle} = \max_k(C_{i,k}^{Active}) - C_{i,j}^{Active}$$

Then, GCoM splits the active cycles of a sub-core into three components, Base, ComData, and MemData, as follows:

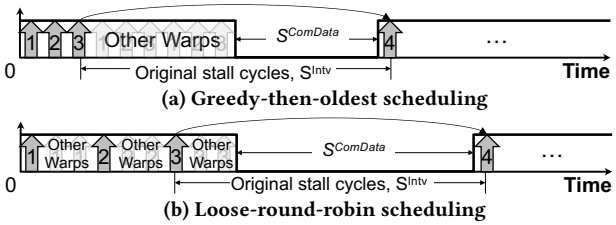
$$C_{i,j}^{Active} = C_{i,j}^{Base} + S_{i,j}^{ComData} + S_{i,j}^{MemData}$$

where $C_{i,j}$ represents the execution cycles of the j-th sub-core of the i-th SM. The Base component indicates the cycles a sub-core spends to execute its allocated warp instructions without any hazards, and the other two components characterize the data hazards on functional unit outputs (i.e., ComData) and long-latency memory accesses (i.e., MemData).

GCoM computes $C_{i,j}^{Base}$ by calculating how many cycles a sub-core would require with an assumption that all instructions take a single cycle and no hazard occurs:

$$C_{i,j}^{Base} = \frac{totalNumWarpInsts_{i,j}}{issueRate}$$
$$totalNumWarpInsts_{i,j} = \#Warps_{i,j} \times NumWarpInsts$$

where $\#Warps_{i,j}$ is the number of warps the j-th sub-core of the i-th SM executes, $NumWarpInsts$ represents the number of the representative warp's instructions, and $totalNumWarpInsts_{i,j}$, in turn, indicates the total number of instructions executed by the sub-core.



(a) Greedy-then-oldest scheduling

(b) Loose-round-robin scheduling

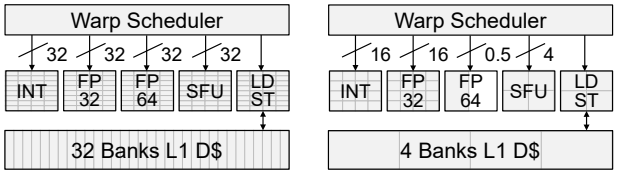
Figure 8: Modeling the ComData stalls with respect to the warp scheduling policy of the per-sub-core warp schedulers

Based on interval analysis, GCoM models the stall cycles, $S_{i,j}^{ComData}$ and $S_{i,j}^{MemData}$, induced by the data dependencies on long-latency operations. As shown in Fig. 8, the execution of a warp consists of multiple intervals separated by the disruptive stall events. GCoM classifies the stall cycles as either $S_{i,j}^{ComData}$ or $S_{i,j}^{MemData}$ depending on whether the stalls are due to memory operations: $S_{i,j}^{MemData}$ if a data dependency on a long-latency memory access exists, $S_{i,j}^{ComData}$ otherwise. Some of the stall cycles get hidden by the instructions of the other concurrently-executing warps due to the highly multi-threaded execution model of GPUs. Therefore, for accurate stall cycle accounting, GCoM needs to estimate how many instructions the other warps can issue during the stalls.

The scheduling policy of the warp schedulers greatly affects the execution order of the warps, and thus the amount of stall cycles which can be hidden. GCoM models two representative warp scheduling policies: greedy-then-oldest (GTO) and loose-round-robin (LRR). The GTO policy executes the instructions of the current warp until it encounters a stall event. Then, it schedules the oldest warp which has a ready instruction (Fig. 8a). GCoM estimates the number of warps and instructions which can be issued during the stall cycles, S_k^{Intv}, and computes the amount of non-hidden stall cycles, $S_{i,j}^{ComData}$, as follows:

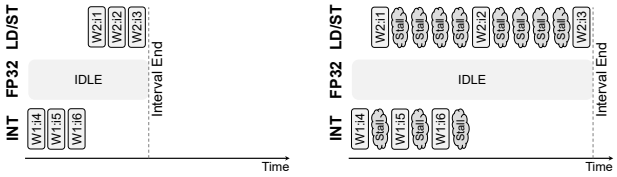
$$S_{i,j}^{ComData} = \sum_{k \in Intervals} max(S_k^{Intv} - Cycles_{other}, 0)$$
$$Cycles_{other} = \frac{P_{warp} \times (\#Warps_{i,j} - 1) \times AvgIntvInsts}{IssueRate}$$
$$P_{warp} = min(S_k^{Intv} \times P_{inst}, 1)$$

where S_k^{Intv} is the stall cycles of the representative warp's k-th interval, $Cycles_{other}$ is the execution cycles of the concurrent warps, and $AvgIntvInsts$ represents the interval's average number of instructions. P_{inst} and P_{warp} denote the instruction and warp issue probabilities, respectively. All the necessary statistics (e.g., $AvgIntvInsts$, P_{inst}) are obtained during the interval profiling stage. If the stall



(a) GPUMech and MDM (b) The NVIDIA Turing GPU

Figure 9: Differences in the numbers of the per-sub-core functional unit lanes and the per-core L1 D$ banks of the existing GPU analytical models and modern GPUs



(a) 32 lanes & L1 D$ banks (b) Limited lanes & L1 D$ banks

Figure 10: The impact of the limited functional unit lanes and L1 D$ banks of modern GPUs on the intra-SM contention

events are due to memory operations, GCoM classifies the cycles as $S_{i,j}^{MemData}$.

The LRR policy, on the other hand, issues the instructions of the ready warps in a round-robin manner. The policy issues one instruction of a warp, then moves on to the next ready warp. In case the warp schedulers employ the LRR policy, only the last instructions of the remaining warps can hide the stall cycles as illustrated in Fig. 8b. As a result, GCoM estimates the number of instructions with the instruction issue probability, P_{inst}, using the following equation:

$$S_{i,j}^{ComData} = \sum_{k \in intervals} S_k^{Intv} - (\#Warps_{i,j} - 1) \times P_{inst}$$

3.4 Modeling the Cycles of a Core

GCoM's intra-SM contention model aims to accurately capture the MemStruct stalls due to the limited L1 D$ banks. As the sub-core model assumes that each sub-core works independently, has enough resources for execution, and does not take into account any intra-SM contentions, GCoM adapts its inter-SM contention model on top of the sub-core model to capture the intra-core load imbalances. The intra-SM contention model computes the execution cycles of the sub-cores, averages the per-sub-core cycles to build the SM's cycle stack, and augments the cycle stack with the stall cycles:

$$C_i^{Active} = \frac{\sum_{j=0}^{numSubCoresSM-1} C_{i,j}}{numSubCoresSM} + S_i$$

where S_i denotes the stall cycles of the i-th SM, and $numSubCoresSM$ is the number of sub-cores per SM.

The intra-SM contention model quantifies the effect of the additional stall cycles induced by the per-core L1 D$ resource contention. The existing GPU analytical models allow a sub-core to execute a load/store instruction on every cycle by assuming a sufficient number of L1 D$ banks. However, in modern GPUs, the L1 D$s have fewer bank counts, and thus incur the MemStruct stalls as shown in Fig. 9 and Fig. 10. To accurately model the MemStruct stalls, along with the ComStruct stalls of the sub-cores, GCoM breaks down S_i into five components:

$$S_i = S_i^{ComStruct} + S_i^{MemStruct} + S_i^{MSHR} + S_i^{NoC} + S_i^{DRAM}$$

Note that the memory-side contention (i.e., S_i^{MSHR}, S_i^{NoC}, S_i^{DRAM}) is estimated by MDM, the memory contention model of GCoM.

As the amount of the ComStruct stalls differs with respect to the number of concurrently-executing warps on a sub-core, GCoM derives $S_i^{ComStruct}$ as:

$$S_i^{ComStruct} = S^{ComStruct}(numCncrWarps) \times \lfloor \frac{numWarps_i}{numCncrWarps} \rfloor + S^{ComStruct}(numWarps_i \mod numCncrWarps)$$

where $numCncrWarps$ is the maximum number of warps that an SM can concurrently execute, and $numWarps_i$ indicates the number of warps the i-th SM has.

GCoM models the structural hazard stalls for an interval as the difference between the cycles a warp scheduler takes to issue all the instructions of the interval with and without taking the limited per-sub-core functional units into account. Accordingly, we get:

$$S^{ComStruct}(x) = \sum_{k \in intervals} \left(C_k^{IssueMax}(x) - C_k^{IssueBase}(x) \right)$$

where x is the number of the concurrently-executing warps of a sub-core, $C_k^{IssueMax}$ is the issue cycles to a functional unit which takes the longest in the k-th interval. $C_k^{IssueBase}$ is the base issue cycles derived by assuming that the functional units have sufficient lanes. These are calculated as:

$$numActSubCores(x) = \min(x, numSubCoresSM)$$

$$C_k^{IssueMax}(x) = \max_{m \in FU} \left\{ C_k^{IssueBase}(x), C_{k,m}^{Issue}(x), C_{k,L1}^{Issue}(x) \right\}$$

$$C_k^{IssueBase}(x) = \frac{I_k \cdot x}{numActSubCores(x) \cdot IssueRate}$$

$$C_{k,m}^{Issue}(x) = \frac{I_m \cdot II_m \cdot x}{numActSubCores(x) \cdot IssueRate} \quad (I_m \leq I_k)$$

$$C_{k,L1}^{Issue}(x) = \lceil \frac{b_k}{B_k^{L1}} \rceil \times x$$

where x is the number of the concurrently-executing warps on a sub-core, I_k is the number of warp instructions in the k-th interval of the representative warp, I_m is the number of warp instructions issued to the m-th functional unit, and II_m denotes the functional unit's initiation interval. $C_{k,m}^{Issue}$ denotes the issue cycles to the m-th functional unit in the k-th interval. $C_{k,L1}^{Issue}$ represents the issue cycles to the L1 D$ in the k-th interval, which we discuss below.

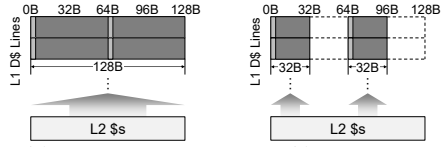
GCoM captures the MemStruct stalls due to the limited L1 D$ banks similar to how it estimates the ComStruct stalls. In case the L1 D$ incurs the largest amount of structural hazards, GCoM classifies the identified structural stalls as MemStruct instead of ComStruct. That is, when $C_{k,L1}^{Issue}(x)$ becomes $C_k^{IssueMax}(x)$, GCoM uses the following equation instead:

$$S^{MemStruct}(x) = \sum_{k \in intervals} \left(C_{k,L1}^{Issue}(x) - C_k^{IssueBase}(x) \right)$$

where b_k is the amount of L1 D$ accesses incurred by the representative warp and B_k^{L1} is the effective L1 D$ bandwidth of the k-th



(a) Four-byte accesses with a 64-byte stride on 128-byte $ lines



(b) Non-sectored (c) Sectored

Figure 11: The behaviors of non-sectored and sectored L1 D$s

Table 3: A summary of GCoM and CPI stack components

Stall	GCoM
Base	C^{Base}
Compute Data (ComData)	$S^{ComData}$
Compute Structural (ComStruct)	$S^{ComStruct}$
Memory Data (MemData)	$S^{MemData} + S^{NoC} + S^{DRAM}$
Memory Structural (MemStruct)	$S^{MemStruct} + S^{MSHR}$
Control/synchronization (Ctrl)	N/A; negligible [25]
Idle	$C_i^{Idle} + C_{i,j}^{Idle}$

interval. Note that GCoM collects all the necessary memory-related information, including the L1 D$ accesses and bandwidth, during the cache simulation stage.

3.5 Modeling Sectored and Streaming L1 Caches

As shown in Table 1, modern GPUs now employ sectored L1 D$ as they can greatly reduce the amount of memory traffic.Unfortunately, the existing GPU analytical models overlook the sectored L1 D$ and overestimate the memory-side contention. Fig. 11 demonstrates the different behaviors of the non-sectored and sectored caches when four threads of the same warp issue 4-byte memory requests (Fig. 11a). Assuming that the caches are cold, the non-sectored cache uses its line size as the memory access granularity and fetches line-sized data from the L2 $s. With the sectored cache, however, all the necessary data can be delivered using only the two sectors of a cache line, resulting in fetching a smaller amount of data from the L2 $s (Fig. 11c). As the behaviors of the caches significantly differ, GCoM needs to accurately model the sectored L1 D$s.

GCoM accurately models the sectored L1 D$s using its cache simulator. The cache simulator supports both sectored and non-sectored L1 D$s. When the provided architectural parameters employ the sectored L1 D$s, the cache simulator issues the sector-sized memory requests, instead of the line-sized ones, to the L2 D$s. After the simulation, the cache simulator passes the per-sector cache statistics (e.g., per-sector hit ratios) to the memory contention model.

The L1 D$s are not only sectored, but also streaming. For modeling the streaming behaviors of the L1 D$s, GCoM takes advantage of MDM [50]. MDM models the lockup-free characteristics of the streaming L1 D$s by assuming a large number of MSHR entries (e.g., 4096 in [50]). GCoM follows MDM's approach and models the lockup-free characteristics by assuming a sufficient number of MSHR entries per L1 D$.

In summary, Table 3 shows how the CPI stack components map to GCoM's analytical expressions. Note that, similar to the existing models, GCoM ignores the front-end stall events (e.g., control hazards) as they are known to incur negligible overheads [25].

Table 4: NVIDIA RTX 2060 GPU configuration

Parameter	Value
Clock Frequency	1,365 MHz
# SMs	30
# Sub-Cores / SM	4
Resources / Sub-Core (II: Initiation interval)	**Warp Scheduler:** 1× single-issue, GTO **INT:** 16×, 2-cycle II, 2 cycles / warp inst. **FP32:** 16×, 2-cycle II, 2 cycles / warp inst. **FP64:** 0.5×, 64-cycle II, 64 cycles / warp inst. **SFU:** 4×, 8-cycle II, 21 cycles / warp inst. **Tensor Core:** 2×, 16-cycle II, 16 cycles / warp inst. **L1 D$:** Sectored, streaming, write-through, 128 B/line, 64 ways, 4 banks, 32 B/sector, 256 MSHRs, 32 merge entries / MSHR
Interconnection Network	Crossbar, 1024 GB/s
# Memory Channels	12
L2 $s	Sectored, streaming, write-back, 194 cycles, 128 B/line, 16 ways, 24 banks, 32 B/sector, 192 MSHRs, 4 merge entries / MSHR
Off-Chip DRAM	96 cycles, GDDR6, 336 GB/s

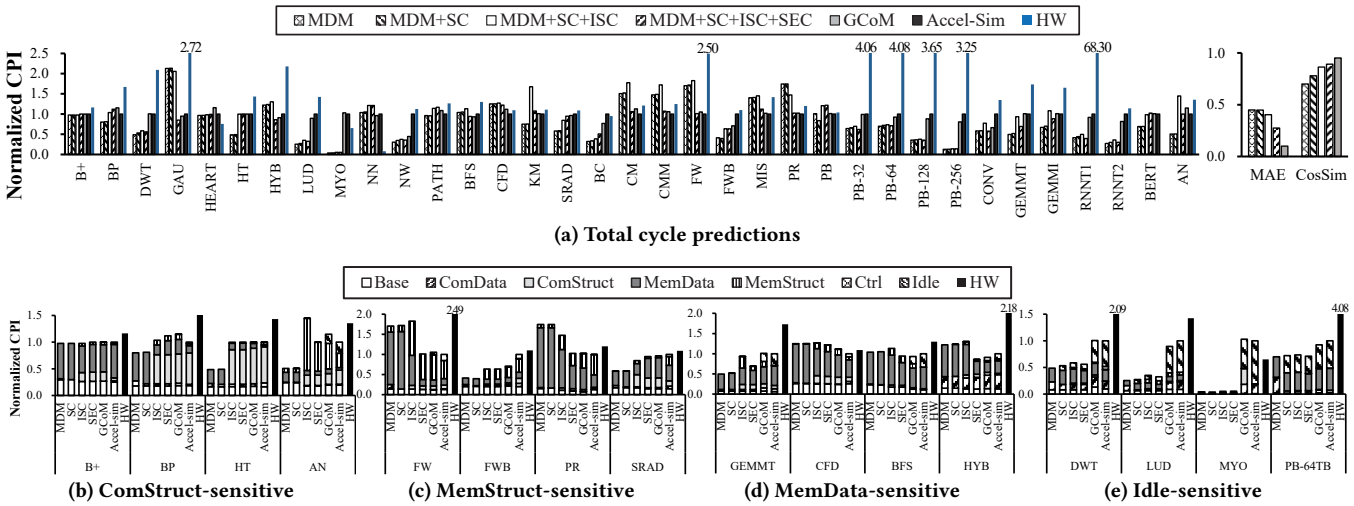
Table 5: Evaluated GPU applications

Benchmark	Workload	# Insts. of the Top-1 Kernel	Abbr.
Rodinia	B+ tree	7,916,078	B+
	Back propagation	1,376,279	BP
	Breadth-first search	8,158,246	BFS
	CFD solver	11,183,315	CFD
	Discrete wavelet transform	1,070,320	DWT
	Gaussian elimination	63,648	GAU
	Heart wall	221,748	HEART
	HotSpot	12,830,168	HT
	Hybrid sort	19,897,711	HYB
	K-means	32,364,672	KM
	K-nearest neighbor	42,840	NN
	LU decomposition	2,198	LUD
	Myocyte	11,492	MYO
	Needleman-Wunsch	88,704	NW
	Pathfinder	2,648,360	PATH
	SRAD	2,177,175	SRAD
Pannotia	Betweenness centrality	1,309,199	BC
	Floyd-Warshall	58,632	FW
	Floyd-Warshall (blocking)	1,171,600	FWB
	Graph coloring (max.)	5,556,737	CM
	Graph coloring (max./min.)	6,161,701	CMM
	Maximal independent set	3,993,907	MIS
	Page rank (spmv)	5,675,887	PR
DeepBench	Convolution (inference)	60,757,908	CONV
	GEMM (train)	1,875,712	GEMMT
	GEMM (infernece)	8,763,136	GEMMI
	Recurrent operatioins (train, lstm)	33,024	RNNT1
	Recurrent operatioins (train, gru)	36,864	RNNT2
MLPerf	BERT (inference)	61,869,497	BERT
Polybench	2D convolution	19,916,288	PB
Tango	AlexNet	53,191,040	AN

4 EVALUATION

4.1 Experimental Setup

To evaluate GCoM, we compare its total cycle predictions and CPI stacks against those of MDM, the state-of-the-art GPU analytical model. For the comparison, we construct reference CPI stacks from Accel-Sim, a detailed cycle-level simulator validated against modern NVIDIA GPUs. We configure Accel-Sim, MDM, and GCoM to model a Turing-based NVIDIA RTX 2060 GPU using the configuration shown in Table 4. As benchmarks, we use 31 representative GPU applications collected from Rodinia [11], Pannotia [10], DeepBench [35], MLPerf [33, 41], Polybench [20], and Tango [29] benchmark suites. The benchmarks cover a wide range of GPU applications including irregular graph processing and large-scale neural networks. Table 5 lists the characteristics of the benchmarks including the warp instruction counts of their longest-running (top-1) kernels.



(a) Total cycle predictions

(b) ComStruct-sensitive (c) MemStruct-sensitive (d) MemData-sensitive (e) Idle-sensitive

Figure 12: The total cycles and the CPI stacks predicted by Accel-Sim, MDM, and GCoM

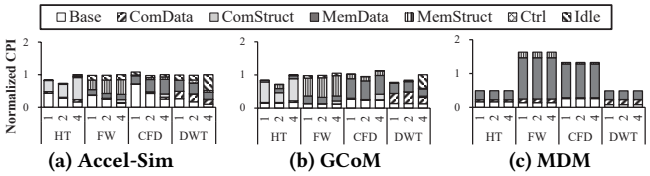
Among the benchmarks, our target NVIDIA RTX 2060 GPU fails to fully execute `BERT` due to its limited memory size [6]. As a workaround, for `BERT`, we collect and use the first 857 kernel invocations, the full set of kernels which run successfully on the GPU without incurring an out-of-memory error. For the DeepBench benchmarks, which we employ to evaluate GCoM's Tensor Core support, we collect and use only the kernels utilizing Tensor Cores. To better demonstrate the need for accurate modeling of the Idle cycles induced by the intra-core load imbalances, we consider four scaled-down variations of `PB`. The scaled-down variations differ in the number of threads per TB: 32, 64, 128, and 256 threads per TB for `PB-32`, `-64`, `-128`, and `-256`, respectively.

4.2 Accurate GPU Performance Modeling

We evaluate the modeling accuracy of GCoM by comparing its total cycle predictions and CPI stacks against those of Accel-Sim and MDM. GCoM should achieve higher modeling accuracy than MDM as it employs detailed GPU core modeling which MDM lacks of. In addition, to analyze the impacts of the key modeling components of GCoM on the modeling accuracy, we incrementally apply GCoM's sub-core model (`+SC`), intra-SM contention model (`+ISC`), sectored L1 D$ model (`+SEC`), and SM load imbalance model (`+SLI`, and thus GCoM) on top of MDM. We employ the mean absolute error (MAE) and cosine similarity as the evaluation metrics.

The results show that GCoM significantly improves the modeling accuracy by achieving an MAE of 10.0% against Accel-Sim in total cycle predictions, whereas MDM suffers from an MAE of 44.9%. Fig. 12a compares the total cycle predictions of MDM, GCoM, and Accel-Sim, and the total cycle counts measured from a real NVIDIA RTX 2060 GPU. The results also show the significance of the three key intra-core stall events GCoM focus on; applying `+SC`, `+ISC`, `+SEC`, and `+SLI` on top of MDM achieve 44.8%, 40.4%, 27.5%, and 10.0% MAEs, respectively. In terms of accurate CPI stack construction, GCoM achieves a cosine similarity of 0.9517 against Accel-Sim, whereas MDM falls short by achieving a cosine similarity of 0.7000.

Aimed at illustrating GCoM's accurate core-side stall modeling, we select a total of 16 benchmarks highly sensitive to the four key core-side stalls GCoM focuses on (i.e., ComStruct, MemStruct, MemData, Idle). Fig. 12b through Fig. 12e show the CPI stacks constructed by Accel-Sim, MDM, and GCoM with the selected benchmarks. First, GCoM accurately captures the ComStruct stalls caused by the limited per-sub-core functional units (Fig. 12b). Second, by taking into account the limited per-core L1 D$ banks, GCoM can reveal the MemStruct stalls (Fig. 12c). Third, Fig. 12d shows that GCoM faithfully takes into account the sectored L1 D$s. Fourth, GCoM correctly identifies the Idle stalls incurred by intra- and inter-core load imbalances as shown in Fig. 12e. The results clearly show that the detailed GPU core modeling of GCoM accurately identifies the key core-side stalls; however, MDM fails to do so due to its outdated and highly abstract GPU core model.



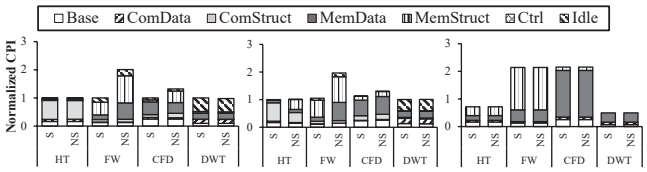
(a) Accel-Sim (b) GCoM (c) MDM

Figure 13: The CPI stacks of Accel-Sim, GCoM, and MDM with varying per-SM sub-core counts: 1, 2, and 4.

4.3 Accurate GPU Design Space Explorations

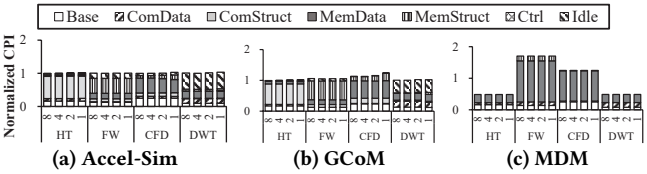
Using its accurate and detailed GPU core model, GCoM can perform various GPU design space explorations. We perform five example design space explorations, using four representative benchmarks primarily bottlenecked by different key core-side stall events. Through the design space explorations, we aim to show that GCoM can provide accurate insights into the performance.

Per-SM Sub-Core Count. We first examine GCoM's ability to accurately model the performance impacts of the number of sub-cores per SM. The number of sub-cores primarily affects the ComStruct and Idle stalls. Decreasing the number of sub-cores per SM can decrease both of the stalls as the per-sub-core functional unit counts



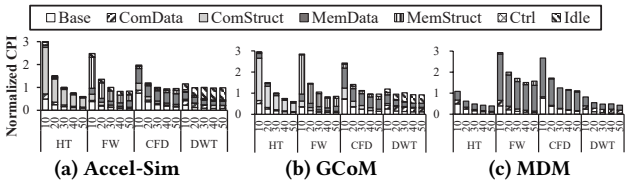
(a) Accel-Sim (b) GCoM (c) MDM

Figure 14: The CPI stacks of Accel-Sim, GCoM, and MDM with sectored (S) and non-sectored (NS) L1 D$s.



(a) Accel-Sim (b) GCoM (c) MDM

Figure 15: The CPI stacks of Accel-Sim, GCoM, and MDM with varying per-core L1 D$ bank counts: 1, 2, 4, and 8.



(a) Accel-Sim (b) GCoM (c) MDM

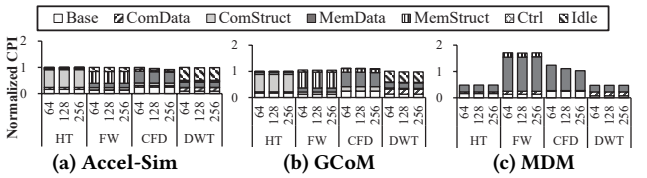
Figure 16: The CPI stacks of Accel-Sim, GCoM, and MDM with varying SM counts: 10, 20, 30, 40, and 50.

increase and the possibility of inter-sub-core load imbalance decreases. When reducing the number of sub-cores per SM, we keep the peak performance as-is by not changing the total functional unit counts and by increasing the issue rate of the warp schedulers.

Fig. 13 shows how the CPI stacks of Accel-Sim, GCoM, and MDM change as we decrease the number of sub-cores per SM from 4 to 1. The results show that GCoM accurately models the decreased ComStruct stalls as the per-core sub-core count decreases; GCoM sustains its high modeling accuracy by achieving an MAE of 4.7% with two sub-cores per SM, and an MAE of 6.9% with one sub-core per SM. However, MDM cannot identify the ComStruct stalls and provides an incorrect performance optimization hints. For example, the primary performance bottlenecks of HT are the ComStruct stalls, whereas MDM reports MemData stalls as a major bottleneck.

Sectored & Non-Sectored L1 D$s. Fig. 14 shows the CPI stacks of Accel-Sim, GCoM, and MDM with sectored and non-sectored L1 D$s. The CPI stacks of Accel-Sim and GCoM accurately capture the changes in the MemData stalls of FW. Unfortunately, MDM does not model the sectored L1 D$s and overestimates the MemData stalls.

Per-Core L1 D$ Bank Count. We now examine how the CPI stacks of Accel-Sim, GCoM, and MDM change with respect to the number of per-core L1 D$ banks. The limited L1 D$ banks can incur contention between the LD/ST units of the sub-cores, and thus MemStruct stalls. Fig. 15 shows that, with FW, GCoM accurately identifies the large MemStruct stalls using its intra-SM contention model. MDM overlooks the MemStruct stalls and provides the CPI stacks which denote the MemData stalls as the key bottleneck.



(a) Accel-Sim (b) GCoM (c) MDM

Figure 17: The CPI stacks of Accel-Sim, GCoM, and MDM with varying per-core L1 D$ sizes: 64, 128, and 256 KB.

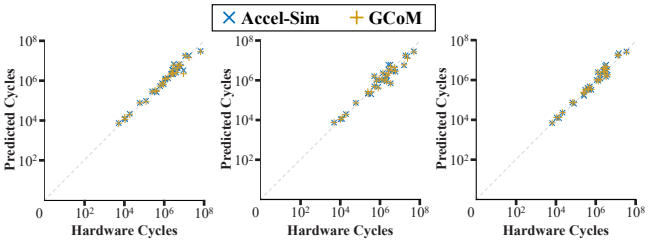
SM Count. We now move on to high-level GPU architectural parameters to demonstrate that GCoM's detailed and accurate GPU core modeling can be helpful to a broader set of design space explorations. In this experiment, we construct the CPI stacks with varying SM counts as choosing an SM count is a major design choice with GPUs. Fig. 16 shows how the CPI stacks of Accel-Sim, GCoM, and MDM differ with varying SM counts. The CPI stacks with DWT show that, as we increase the SM count, the Idle stalls significantly increase due to the worsening inter-SM load imbalance. GCoM correctly captures the increasing Idle stalls, but MDM cannot as it assumes that all the SMs are fully occupied and active.

Per-Core L1 D$ Size. The per-core L1 D$ size is also an important design choice with GPUs. We examine how the changes in the L1 D$ size affect the performance by changing the way counts of the L1 D$s. Increasing the way counts results in having larger L1 D$ sizes which can greatly reduce the MemData stalls. Fig. 17 shows the CPI stacks with 64-, 128-, and 256-KB L1 D$s. The CPI stacks show that, with the evaluated benchmarks, increasing the L1 D$ size has marginal impacts on the performance. GCoM accurately captures the severe MemStruct stalls, providing an accurate insight that the L1 D$ size may have a marginal impact on the performance. However, MDM's CPI stacks for FW show that the MemData stalls are the key performance bottleneck, hinting that increasing the L1 D$ sizes may greatly benefit the benchmarks.

4.4 Validation Against Real GPU Hardware

For a GPU analytical model to be used in real-world design space explorations, the model should faithfully capture the performance characteristics of a real GPU. In this sense, we validate GCoM against a real NVIDIA RTX 2060 GPU. As the validation involves a single design point, we configure both GCoM and Accel-Sim to use the low-level SASS dynamic instruction traces. We use Nsight Compute CLI [37] to obtain the `gpc__cycles_elapsed.avg` value which tracks the elapsed cycles of a kernel. For each benchmark, we repeat the measurement for 20 iterations and take the average.

Fig. 18a shows the correlations of GCoM and Accel-Sim against the real GPU with the 31 benchmarks. The results show that GCoM achieves an MAE of 24.4% against the real GPU, which is lower than that of Accel-Sim's MAE of 26.7%. Overall, the cycle count correlations of GCoM and Accel-Sim against the RTX 2060 GPU follow a similar trend. This is due to the limited amount of the publicly-available information on the core microarchitectures of modern GPUs; GCoM heavily relies on and models the GPU core microarchitecture assumed by Accel-Sim. In summary, GCoM achieves comparable modeling accuracy against the real GPU and is orders of magnitude faster than Accel-Sim cycle-level simulator.



(a) RTX 2060 (b) RTX 2060 Super (c) RTX 3070

Figure 18: The cycle count correlations of Accel-Sim and GCoM against real NVIDIA GPUs using the SASS traces collected from an NVIDIA RTX 2060 GPU

5 DISCUSSIONS

5.1 Using a GPU's Traces to Model Other GPUs

GCoM takes as input the dynamic execution trace of an application collected from a target GPU through instrumentation. As the trace collection process is time-consuming, it is desirable for GCoM to be able to accurately model different GPUs using the traces collected from one GPU. In this sense, we evaluate how accurately can GCoM model different NVIDIA GPUs using the traces collected from the NVIDIA RTX 2060 GPU. We select NVIDIA RTX 2060 Super and RTX 3070 GPUs as modeling targets. RTX 2060 and RTX 2060 Super share the same Turing microarchitecture, but RTX 3070 is based on the Ampere microarchitecture (see Table 1 for the differences in SM microarchitectures). For all the benchmarks, we configure both GCoM and Accel-Sim to take as input the SASS traces collected from RTX 2060 and the configuration files reflecting the characteristics of the two target GPUs. We then compare the total cycle predictions of GCoM and Accel-Sim against the total cycle counts measured from the two target GPUs.

Fig. 18b and Fig. 18c show the cycle count correlations of GCoM and Accel-Sim against the target GPUs. The results show that, with the RTX 2060 traces, GCoM achieves an MAE of 37.6% against RTX 2060 Super and an MAE of 25.5% against RTX 3070. Accel-Sim achieves 42.8% and 28.9% against RTX 2060 Super and RTX 3070, respectively. Given the MAEs of GCoM and Accel-Sim against the baseline RTX 2060 (GCoM: 24.4%, Accel-Sim: 26.7%), we claim that the additional errors introduced by using the RTX 2060 traces are acceptable and can be mitigated by using the SASS traces collected from the target GPUs, not those collected from RTX 2060.

5.2 Extending GCoM to Model Other GPUs

In this work, we mainly focused on modeling modern NVIDIA GPUs with GCoM; however, one may seek to employ GCoM for modeling GPUs from other vendors which have different core microarchitectures. For example, AMD Graphics Core Next (GCN) GPUs group 64 threads, instead of 32 threads, into a warp and employ 16-wide vector units [1]. GCoM can directly support most of the differences (e.g., set the warp size as 64 for modeling 64-thread warps, model the 16-wide vector units as 16 scalar units).

GCoM can be extended to approximate the behaviors of explicit data dependency resolution instructions (e.g., the `s_waitcnt` instruction of AMD GPUs) by associating *virtual* data dependencies to the instructions. When GCoM encounters such an instruction during the trace collection stage, it can add virtual data dependencies between the instruction and the previous instructions which must be completed to satisfy the instruction's requirements. More accurate modeling of the instructions would require SM-wide synchronization and functional state modeling (e.g., track the value of the SM-wide `VM_CNT` counter), which we leave as future work.

In case we lack dynamic trace instrumentation tools, GCoM can use the functional and/or timing simulators implementing the target GPU microarchitectures and instruction sets (e.g., MGPUSim[46] for AMD GPUs) to collect dynamic traces.

6 RELATED WORK

6.1 GPU Analytical Modeling

Prior studies have proposed analytical modeling for GPUs, but they only model specific components [31, 36] or have limited applicability due to their lack of detailed GPU core modeling. Nugteren et al. [36] proposed a reuse distance-based GPU cache model. Kiani and Rajabzadeh took a similar approach to model GPU caches [31]. Baghsorkhi et al. [7] introduced work flow graph, which augments the control flow graph of a GPU kernel with explicit memory operation nodes and data dependence edges. The work flow graph embeds the instruction and memory latency of a particular GPU architecture in the edge weights via static analysis with architecture specification. Hong and Kim [24] proposed a model that estimates CPI based on the estimated amount of memory-level parallelism. Sim et al. [44] extended the model and developed GPUPerf, a GPU performance profiler, to pinpoint performance bottlenecks. Zhang and Owens [56] proposed a microbenchmark-based model which estimates the execution time with the instruction pipeline, shared memory accesses, and global memory accesses. Huang et al. proposed GPUMech [25], the first GPU interval model. Based on GPUMech, Heo et al. [23] proposed a GPU performance model that predicts the execution time of a single DNN layer. However, none of the proposed models, including the state-of-the-art MDM [50, 51], considers the key intra-core stall cycles which GCoM focuses on, leading to high modeling errors with modern GPU designs.

6.2 Simulation-Based Performance Evaluation

Detailed cycle-level simulation is the de-facto standard for GPU architecture evaluation, but its slow simulation speed makes it inappropriate for exploring design space at the early design phase. One way of reducing the simulation time is to reduce the size of target workloads. Yu et al. [54, 55] developed a method to synthesize a miniature that mimics a full-size GPU kernel for performance evaluation. Wang et al. [52] proposed a static analysis-based approach to generate workload traces which embed the branch and cache behaviors. TBPoint [26] and Principal Kernel Analysis (PKA) [6] can be used to identify and simulate only a representative portion of a workload. Especially, PKA is a fully-automated inter- and intra-kernel reduction scheme and has been extensively validated with representative large-scale workloads (e.g., MLPerf [33, 41]) and against NVIDIA Turing and Ampere GPUs. These studies are orthogonal and can be equally applied to GCoM. Reducing the workload sizes reduces the sizes of the traces GCoM takes as input, allowing GCoM to incur smaller one-time and recurring costs and accelerate its analytical modeling speed.

Other studies propose to combine multiple models with different details, from analytical models to cycle-level detailed simulations. Villa et al. [48] designed a GPU simulator that uses different abstract levels for each GPU component; for instance, they fully model the instruction dependencies for SM pipelines while using simple latency plus bandwidth models for on-chip interconnect channels. Similarly, PAQSim [19] replaces some detailed simulations with analytical models to accelerate graphics workload simulations. PPT-GPU [3, 4] exploits reuse distance-based analytical models for memory systems. For scalability, it simulates only a small number of active TBs in detail and reuses the result for the remaining blocks. Contrary to the simulation-based approaches, our scheme GCoM is of analytical modeling and takes negligible recurring costs for evaluating different designs.

6.3 ML-Based Performance Evaluation

Machine learning (ML)-based approaches build performance models without microarchitecture knowledge. Instead, they require hundreds of static or runtime execution statistics captured from performance counters or simulations for training the models. Dublish et al. [13] and Ardalani et al. [5] used regression to predict the performance of GPU architecture. Braun et al. [8] and O'neal et al. [40] proposed a model based on random forests. Wu et al. [53] exploited neural networks, and Guerreiro et al. [21] developed a recurrent neural network-based model, which takes the sequence of PTX instructions. However, ML-based models only predict the end-to-end performance metrics (e.g., IPC) and do not provide a deep understanding of GPU architecture designs. Our scheme GCoM is built upon the understanding of GPU architecture and provides detailed insights to identify bottlenecks.

7 CONCLUSION

We proposed GCoM, a detailed and accurate GPU core model which faithfully captures the key core-side stall events of modern GPUs. GCoM targets detailed modeling of the core microarchitectures of modern GPUs, which have been overlooked by the existing GPU analytical models. Our evaluation of GCoM against Accel-Sim cycle-level simulator and a real GPU shows that GCoM greatly improves the modeling accuracy, whereas the existing models suffer from high modeling errors due to their lack of detailed GPU core model.

ACKNOWLEDGMENTS

This work was partly supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (2020R1F1A1069742, 2022R1C1C1008131, 2022R1C1C1011307, 2021 R1F1A1062902), Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (2021-0-00853, 2020-0-01361, 2021-0-02051), and the Yonsei Signature Research Cluster Program (2022-22-0002). Jounghoo Lee, Yeonan Ha, and Suhyun Lee are with the Department of Computer Science at Yonsei University and are partly supported by the BK21 FOUR (Fostering Outstanding Universities for Research) funded by the Ministry of Education (MOE) of Korea and National Research Foundation (NRF) of Korea.

REFERENCES

[1] Advanced Micro Devices, Inc. 2016. RADEON: Dissecting the Polaris Architecture. https://www.amd.com/system/files/documents/polaris-whitepaper.pdf.
[2] Johnathan Alsop, Matthew D. Sinclair, Rakesh Komuravelli, and Sarita V. Adve. 2016. GSI: A GPU Stall Inspector to Characterize the Sources of Memory Stalls for Tightly Coupled GPUs. In *Proc. 2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.
[3] Yehia Arafa, Abdel-Hameed Badawy, Ammar ElWazir, Atanu Barai, Ali Eker, Gopinath Chennupati, Nandakishore Santhi, and Stephan Eidenbenz. 2021. Hybrid, scalable, trace-driven performance modeling of GPGPUs. In *Proc. 2021 International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
[4] Yehia Arafa, Abdel-Hameed A. Badawy, Gopinath Chennupati, Nandakishore Santhi, and Stephan Eidenbenz. 2019. PPT-GPU: Scalable GPU Performance Modeling. *IEEE Computer Architecture Letters (CAL)* 18 (2019).
[5] Newsha Ardalani, Clint Lestourgeon, Karthikeyan Sankaralingam, and Xiaojin Zhu. 2015. Cross-Architecture Performance Prediction (XAPP) Using CPU Code to Predict GPU Performance. In *Proc. 48th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
[6] Cesar A. Baddouh, Mahmoud Khairy, Roland Green, Mathias Payer, and Timothy G. Rogers. 2021. Principal Kernel Analysis: A Tractable Methodology to Simulate Scaled GPU Workloads. In *Proc. 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
[7] Sara S. Baghsorkhi, Matthieu Delahaye, Sanjay J. Patel, William D. Gropp, and Wen mei W. Hwu. 2010. An Adaptive Performance Modeling Tool for GPU Architectures. In *Proc. 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*.
[8] Lorenz Braun, Sotirios Nikas, Chen Song, Vincent Heuveline, and Holger Fröning. 2020. A Simple Model for Portable and Fast Prediction of Execution Time and Power Consumption of GPU Kernels. *ACM Transactions on Architecture and Code Optimization (TACO)* 18 (2020).
[9] John Burgess. 2020. RTX on—The NVIDIA Turing GPU. *IEEE Micro* 40 (2020).
[10] Shuai Che, Bradford M. Beckmann, Steven K. Reinhardt, and Kevin Skadron. 2013. Pannotia: Understanding irregular GPGPU graph applications. In *Proc. 2013 IEEE International Symposium on Workload Characterization (IISWC)*.
[11] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W, Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *Proc. 2009 IEEE International Symposium on Workload Characterization (IISWC)*.
[12] Jack Choquette, Olivier Giroux, and Denis Foley. 2018. Volta: Performance and Programmability. *IEEE Micro* 38 (2018).
[13] Saumay Dublish, Vijay Nagarajan, and Nigel Topham. 2019. Poise: Balancing Thread-Level Parallelism and Memory System Performance in GPUs Using Machine Learning. In *Proc. 25th IEEE International Symposium on High Performance Computer Architecture (HPCA)*.
[14] Stijin Eyerman, Lieven Eeckhout, Tejas Karkhanis, and James E. Smith. 2009. A Mechanistic Performance Model for Superscalar Out-of-Order Processors. *ACM Transactions on Computer Systems* 27 (2009).
[15] B.A. Fields, R. Bodik, M.D. Hill, and C.J. Newburn. 2003. Using interaction costs for microarchitectural bottleneck analysis. In *Proc. 36th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
[16] B. Fields, S. Rubin, and R. Bodik. 2001. Focusing processor policies via critical-path prediction. In *Proc. 28th IEEE/ACM International Symposium on Computer Architecture (ISCA)*.
[17] Denis Foley and John Danskin. 2017. Ultra-Performance Pascal GPU and NVLink Interconnect. *IEEE Micro* 37 (2017).
[18] Michael Garland, Scott Le Grand, John Nickolls, Joshua Anderson, Jim Hardwick, Scott Morton, Everett Phillips, Yao Zhang, and Vasily Volkov. 2008. Parallel Computing Experiences with CUDA. *IEEE Micro* 28 (2008).
[19] Xiang Gong, Chunling HU, and Chu-Cheow Lim. 2020. PAQSIM: Fast Performance Model for Graphics Workload on Mobile GPUs. In *Proc. 21st ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*.
[20] Scott Grauer-Gray, Lifan Xu, Robert Searles, Sudhee Ayalasomayajula, and John Cavazos. 2012. Auto-tuning a bhigh-level language targeted to GPU codes. In *2012 Innovative Parallel Computing (InPar)*.
[21] João Guerreiro, Aleksandar Ilic, Nuno Roma, and Pedro Tomás. 2019. GPU Static Modeling Using PTX and Deep Structured Learning. *IEEE Access* 7 (2019).
[22] Anthony Gutierrez, Bradford M. Beckmann, Alexandru Dutu, Joseph Gross, Michael LeBeane, John Kalamatianos, Onur Kayiran, Matthew Poremba, Brandon Potter, Sooraj Puthoor, Matthew D. Sinclair, Mark Wyse, Jieming Yin, Xianwei Zhang, Akshay Jain, and Timothy Rogers. 2018. Lost in Abstraction: Pitfalls of Analyzing GPUs at the Intermediate Language Level. In *Proc. 24th IEEE International Symposium on High Performance Computer Architecture (HPCA)*.
[23] Seonyeong Heo, Sungjun Cho, Youngsok Kim, and Hanjun Kim. 2020. Real-Time Object Detection System with Multi-Path Neural Networks. In *Proc. 26th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*.

[24] Sunpyo Hong and Hyesoon Kim. 2009. An Analytical Model for a GPU Architecture with Memory-level and Thread-level Parallelism Awareness. In *Proc. 36th IEEE/ACM International Symposium on Computer Architecture (ISCA)*.

[25] Jen-Cheng Huang, Joo Hwan Lee, Hyesoon Kim, and Hsien-Hsin S. Lee. 2014. GPUMech: GPU Performance Modeling Techniques based on Interval Analysis. In *Proc. 47th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[26] Jen-Cheng Huang, Lifeng Nai, Hyesoon Kim, and Hsien-Hsin S. Lee. 2014. TBPoint: Reducing Simulation Time for Large-Scale GPGPU Kernels. In *Proc. 28th IEEE International Parallel and Distributed Processing Symposium (IPDPS)*.

[27] Hanhwi Jang, Jae-Eon Jo, Jaewon Lee, and Jangwoo Kim. 2018. RpStacks-MT: A High-throughput Design Evaluation Methodology for Multi-core Processors. In *Proc. 51st IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[28] Tejas S. Karkhanis and James E. Smith. 2004. A First-Order Superscalar Processor Model. In *Proc. 31st IEEE/ACM International Symposium on Computer Architecture (ISCA)*.

[29] Aajna Karki, Chethan Palangotu Keshava, Spoorthi Mysore Shivakumar, Joshua Skow, Goutam Madhukeshwar Hegde, and Hyeran Jeon. 2019. Detailed Characterization of Deep Neural Networks on GPUs and FPGAs. In *Proc. 12th Workshop on General Purpose Processing Using GPUs (GPGPU)*.

[30] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling. In *Proc. 47th IEEE/ACM International Symposium on Computer Architecture (ISCA)*.

[31] Mohsen Kiani and Amir Rajabzadeh. 2018. Efficient Cache Performance Modeling in GPUs Using Reuse Distance Analysis. *ACM Transactions on Architecture and Code Optimization (TACO)* 15 (2018).

[32] Jaewon Lee, Hanhwi Jang, and Jangwoo Kim. 2014. RpStacks: Fast and Accurate Processor Design Space Exploration Using Representative Stall-Event Stacks. In *Proc. 47th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[33] Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debo Dutta, Udit Gupta, Kim Hazelwood, Andy Hock, Xinyuan Huang, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St John, Carole-Jean Wu, Lingjie Xu, Cliff Young, and Matei Zaharia. 2020. MLPerf Training Benchmark. In *Proc. Machine Learning and Systems 2 (MLSys)*.

[34] Aaftab Munshi. 2009. The OpenCL specification. In *Proc. IEEE Hot Chips 21 Symposium (HCS)*.

[35] Sharan Narang. 2016. DeepBench. https://svail.github.io/DeepBench/.

[36] Cedric Nugteren, Gert-Jan van den Braak, Henk Corporaal, and Henri Bal. 2014. A Detailed GPU Cache Model Based on Reuse Distance Theory. In *Proc. 20th IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

[37] NVIDIA Corporation. 2020. Nsight Compute CLI.

[38] NVIDIA Corporation. 2021. NVIDIA Ampere GA102 GPU Architecture. https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf.

[39] NVIDIA Corporation. 2021. Parallel Thread Execution ISA: Application Guide (v7.4).

[40] Kenneth O'neal, Philip Brisk, Ahmed Abousamra, Zack Waters, and Emily Shriver. 2017. GPU Performance Estimation Using Software Rasterization and Machine Learning. *ACM Trans. Embed. Comput. Syst.* 16 (2017).

[41] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. In *Proc. 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*.

[42] Timothy G. Rogers, Mike O'Connor, and Tor M. Aamodt. 2012. Cache-Conscious Wavefront Scheduling. In *Proc. 45th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[43] Ali G. Saidi, Nathan L. Binkert, Steven K. Reinhardt, and Trevor Mudge. 2008. Full-System Critical Path Analysis. In *Proc. 2008 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.

[44] Jaewoong Sim, Aniruddha Dasgupta, Hyesoon Kim, and Richard Vuduc. 2012. A Performance Analysis Framework for Identifying Potential Benefits in GPGPU Applications. In *Proc. 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*.

[45] John E. Stone, David Gohara, and Guochun Shi. 2010. OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems. *Computing in Science & Engineering* 12 (2010).

[46] Yifan Sun, Trinayan Baruah, Saiful A. Mojumder, Shi Dong, Xiang Gong, Shane Treadway, Yuhui Bao, Spencer Hance, Carter McCardwell, Vincent Zhao, Harrison Barclay, Amir Kavyan Ziabari, Zhongliang Chen, Rafael Ubal, José Abellán, John Kim, Ajay Joshi, and David Kaeli. 2019. MGPUSim: Enabling Multi-GPU Performance Modeling and Optimization. In *Proc. 46th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*.

[47] Teruo Tanimoto, Takatsugu Ono, Koji Inoue, and Hiroshi Sasaki. 2017. Enhanced Dependence Graph Model for Critical Path Analysis on Modern Out-of-Order Processors. *IEEE Computer Architecture Letters (CAL)* 16 (2017).

[48] Oreste Villa, Daniel Lustig, Zi Yan, Evgeny Bolotin, Yaosheng Fu, Niladrish Chatterjee, Nan Jiang, and David Nellans. 2021. Need for Speed: Experiences Building a Trustworthy System-Level GPU Simulator. In *Proc. 27th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*.

[49] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. 2019. NVBit: A Dynamic Binary Instrumentation Framework for NVIDIA GPUs. In *Proc. 52nd IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[50] Lu Wang, Magnus Jahre, Almutaz Adileh, and Lieven Eeckhout. 2020. MDM: The GPU Memory Divergence Model. In *Proc. 53rd IEEE/ACM International Symposium on Microarchitecture (MICRO)*.

[51] Lu Wang, Magnus Jahre, Almutaz Adileh, Zhiying Wang, and Lieven Eeckhout. 2019. Modeling Emerging Memory-Divergent GPU Applications. *IEEE Computer Architecture Letters (CAL)* 18 (2019).

[52] Xiebing Wang, Kai Huang, Alois Knoll, and Xuehai Qian. 2019. A Hybrid Framework for Fast and Accurate GPU Performance Estimation through Source-Level Analysis and Trace-Based Simulation. In *Proc. 25th IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

[53] Gene Wu, Joseph L. Greathouse, Alexander Lyashevsky, Nuwan Jayasena, and Derek Chiou. 2015. GPGPU performance and power estimation using machine learning. In *Proc. 21st IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

[54] Zhibin Yu, Lieven Eeckhout, Nilanjan Goswami, Tao Li, Lizy John, Hai Jin, and Chengzhong Xu. 2013. Accelerating GPGPU Architecture Simulation. In *Proc. ACM SIGMETRICS/International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*.

[55] Zhibin Yu, Lieven Eeckhout, Nilanjan Goswami, Tao Li, Lizy K John, Hai Jin, Chengzhong Xu, and Junmin Wu. 2015. GPGPU-MiniBench: Accelerating GPGPU Micro-Architecture Simulation. *IEEE Trans. Comput.* 64 (2015).

[56] Yao Zhang and John D. Owens. 2011. A Quantitative Performance Analysis Model for GPU Architectures. In *Proc. 17th IEEE International Symposium on High Performance Computer Architecture (HPCA)*.